

Evolving a Testsuite For Shading and Rendering

Daniel Greenstein
Sony Pictures Imageworks
Culver City, CA, USA
greenstein@imageworks.com

Lee Kerley
Sony Pictures Imageworks
Culver City, CA, USA
lee.kerley@gmail.com



Figure 1: Prospect Park Set From Spider-Man: Across the Spider-Verse - Test Setup. © 2022 CTMG, Inc. All Rights Reserved

ABSTRACT

This talk will present some of the software testing methodologies used within the back-end divisions (rendering, shading etc) of Sony Pictures Imageworks. A key focus will be the renderingtestsuite, an image-based testing system. We will delve into its evolution, and discuss the challenges of large-scale image-based testing.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation.**

KEYWORDS

Software Engineering, Software Testing and Validation, Image Comparison

ACM Reference Format:

Daniel Greenstein and Lee Kerley. 2024. Evolving a Testsuite For Shading and Rendering. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641233.3664350>

1 INTRODUCTION

Sony Pictures Imageworks is an Academy Award®-winning visual effects and animation studio known for photoreal live-action visual effects, dynamic creature and character animation and full-CG features.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH Talks '24, July 27–August 01, 2024, Denver, CO, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0515-1/24/07

<https://doi.org/10.1145/3641233.3664350>

As a trailblazer in the industry, we distinguish ourselves by developing many of the software components for digital filmmaking in-house. Noteworthy among these are our award-winning renderer, Arnold [Kulla et al. 2018], our Open Shading Language (OSL) [Gritz et al. 2010] based shader library and material authoring tools, and our Katana based ecosystem of lighting and look development tools.

Developing software for a dynamic and fast-paced film production environment requires rigorous and automated testing systems to catch crashes and look issues before they hit the production floor.

2 IMAGE BASE TESTING

Our rendering software testing methodology hinges on a simple principle: comparing two images. First, we render a "test" image using a pre-release version of our software. We also have stored "reference" images generated with a previously qualified version. We generate a "difference" image between these two image sets. If the pixel values match perfectly, the difference image is black, indicating no undesirable changes. However, any visible data prompts a detailed examination to ensure modifications are intentional and desirable.

3 EARLY DAYS

The genesis of the renderingtestsuite dates back to the late 2000s, during a period of significant transformation in rendering practices at Imageworks. In those days, our testing system comprised mostly of ".ass" files (Arnold's scene description file format) and reference image files that were stored in the source control repository.

While a system like this is the corner stone of many software projects that deal with image generation, and works well for unit testing of specific renderer features. It is not adequate as a full integration test suite that tests the renderer, the shaders, our lighting and lookdev software and other ancillary packages such as our procedural hair generator.



Figure 2: The web view shows you the test results with visual aids to help interpret them

4 MODERNIZING THE SYSTEM

In 2019, we embarked on a mission to modernize image-based testing at Imageworks.

New features we now support are:

- (1) Using "live" production scenes.
- (2) Tests are dynamic, they behave like any other artist submitted job on our render farm but using pre-release dev code injected into a production environment.
- (3) Adding test cases is fast and easy.
- (4) Fast test launching time. Once we see a failure we can abort the rest of the test run.
- (5) Easy access to test environments for debugging.
- (6) Automatic reference generation. Removing the human burden, and potential error.
- (7) DCC agnostic. Test plugin support for Katana, Nuke, Maya.
- (8) Data management is easy.

A core realization drove our approach: rather than thinking about image testing in terms of a "test" and "reference" image, we shifted our mindset towards testing two (A/B) software environments comprising of versioned software components.

The system is divided into several separate sub components:

- (1) A job scheduler that submits the tests to the render farm.
- (2) A plugin system responsible for rendering images from different software packages.
- (3) A "diff engine" that calculates the difference of two sets of images, and records the results in a database.
- (4) A webpage designed to display the results for easy examination and evaluation.

When tests are launched two (A/B) distinct software environments are defined, the system generates a unique hash to identify these environments using the software versions. The hashes can then be used to query the database to see if this environment has already been tested, in which case no new renders are necessary. Concretely, this means that when we submit a "test" vs "reference" style test run, we are only running the "test" side tests. This is entirely automatic. Care is also taken if a test scene is using "live" production data, which can evolve over time, in that circumstance both sides of the test are always re-rendered to ensure no test failures are introduced by data differences.

Tests are automatically launched "nightly" as a general progress check using the head of "main" for the shader library. Tests are also

launched for each new release of major software component. Tests can also be grouped in categories, allowing for faster feedback e.g. only hair tests are run if hair procedural development is undertaken. Artists can also launch tests to make sure hero assets don't suffer any undesirable look changes.

A system like this is very data intensive. For instance, the test data for "Spider-Man: Across the Spider-Verse" totaled 15 terabytes, but our design allows for easy management of this data, test images can be easily re-created from recorded software environments.

5 LIMITATIONS/FUTURE WORK

One of the main challenges of image-based testing lies in interpreting the test results. Mainly dealing with issues of "false negatives", images that are mathematically different from one another but qualitatively identical.

This can be countered in two ways:

- (1) Setting thresholds, we could allow a certain amount of different pixels, or consider certain small enough values as a pass.
- (2) Employing perceptual diffs, such as perceptualdiff [Yee and Newman 2004].

The renderingtestsuite currently only reports differences in images, but there are other metrics available from a render process that we could record and track differences of, such as the number of meshes that were displaced or the memory used during the render. Changes in these metrics could also be regressions introduced by software changes, and it would be nice to capture this in testing.

We are currently integrating our new software packaging system, SPK - <https://getspk.io/>. SPK perfectly encapsulates and tracks different execution environments so it lends itself perfectly to the philosophy behind the renderingtestsuite.

6 CONCLUSION

Detecting software issues before they impact the production process is crucial for studios involved in large-scale film production. At Sony Pictures Imageworks, image-based testing serves as a cornerstone of our software testing approach. Having a scalable solution that operates largely automatically has allowed the developers at Imageworks to focus on development and also increased the confidence with which a production can adopt new software.

ACKNOWLEDGMENTS

This work originated during the late 2000s, when it was led by John Monos. The initial developers, Derek Haase and Juan Carlos Moreno, laid its foundation. The renderingtestsuite heavily leverages OpenImageIO.

REFERENCES

- Larry Gritz, Clifford Stein, Chris Kulla, and Alejandro Conty. 2010. Open Shading Language. In *ACM SIGGRAPH 2010 Talks* (Los Angeles, California) (SIGGRAPH '10). Association for Computing Machinery, New York, NY, USA, Article 33, 1 pages. <https://doi.org/10.1145/1837026.1837070>
- Christopher Kulla, Alejandro Conty, Clifford Stein, and Larry Gritz. 2018. Sony Pictures Imageworks Arnold. *ACM Trans. Graph.* 37, 3, Article 29 (aug 2018), 18 pages. <https://doi.org/10.1145/3180495>
- Yangli Hector Yee and Anna Newman. 2004. A perceptual metric for production testing. In *ACM SIGGRAPH 2004 Sketches* (Los Angeles, California) (SIGGRAPH '04). Association for Computing Machinery, New York, NY, USA, 121. <https://doi.org/10.1145/1186223.1186374>